

Arquitetura

Enterprise (PT-2)

Arquitetura de Sistemas com IA · Módulo 5.2

Prof. Ahirton Lopes, Ph.D.

Rastreamento: **Versão, Diff, Replay**

01

Prompt de produção muda — e ninguém sabe pra qual versão

Sem versionamento, um ajuste de prompt em produção é indistinguível de um bug de modelo: os dois mudam o comportamento sem deixar rastro de qual mudou o quê.

02

Versionar prompt como versiona código

PromptLayer trata cada mudança como um commit — diff visível antes de salvar, histórico completo, e um botão de replay que reexecuta uma requisição passada com uma versão de prompt diferente.

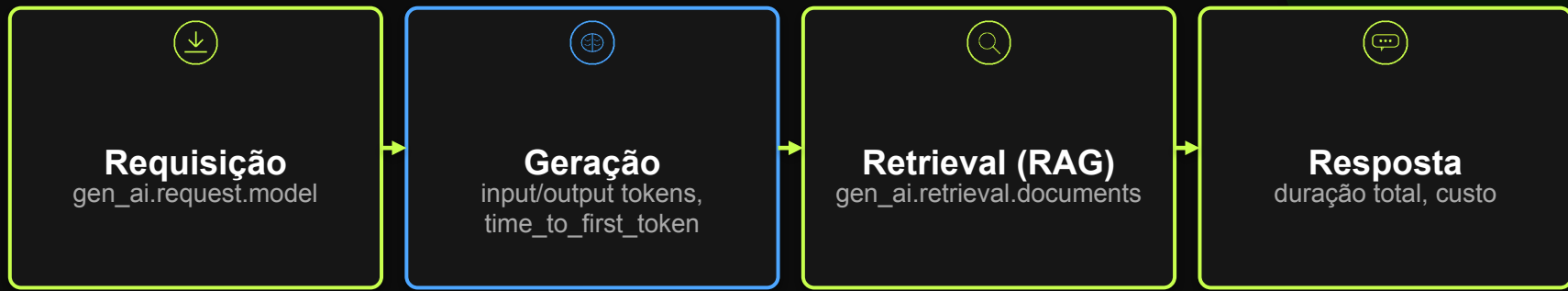
03

A categoria inteira já é grande o bastante pra consolidar

Langfuse, projeto de código aberto, foi adquirido pela ClickHouse em janeiro de 2026 — usado por 19 das 50 maiores empresas da Fortune 500, incluindo Intuit, Twilio e Merck, com mais de 23 milhões de instalações de SDK por mês.

Arize Phoenix (open source, sobre OpenInference/OpenTelemetry): opção self-hosted, sem depender de serviço externo — relevante para compliance com dado sensível de paciente.

O Que Logar: **Metadado Que Importa**



OpenTelemetry GenAI Semantic Conventions: padrão aberto (ainda em desenvolvimento) que nomeia esses campos — `gen_ai.usage.input_tokens`, `gen_ai.server.time_to_first_token` — pra qualquer ferramenta de observabilidade ler do mesmo jeito.

Ocupado ou **Quebrando?**

01

Os quatro sinais que toda arquitetura enterprise já mede

Latência, Tráfego, Erros e Saturação — formalizados no livro Site Reliability Engineering do Google (capítulo de Rob Ewaschuk). Servem pra qualquer sistema distribuído, incluindo o seu.

02

Mas os quatro sinais podem estar todos verdes e o modelo já ter piorado

Chen, Zaharia e Zou (Stanford/Berkeley, 2023) mediram a acurácia do GPT-4 em identificar números primos caindo de 84% pra 51% entre março e junho do mesmo ano — mesma API, mesma latência, comportamento completamente diferente.

03

Isso já aconteceu em produção, reconhecido pelo próprio fornecedor

Em dezembro de 2023, usuários relataram o GPT-4 “preguiçoso” — respostas truncadas, código incompleto. A OpenAI confirmou publicamente: “não atualizamos o modelo desde 11 de novembro, e isso certamente não é intencional.” Nenhum dos quatro sinais clássicos capturou isso.

DPD (jan/2024): usuário frustrado manipulou o chatbot de atendimento a xingar a própria empresa e escrever um poema detonando o serviço — API respondendo rápido, sem erro, os quatro sinais clássicos todos verdes durante a conversa inteira. Caso viralizou no X, coberto por BBC e Fortune.

Guardrail: Pegar Antes, Não Só Depois

Observabilidade mede depois — um guardrail decide antes

- 01** A taxa de rejeição no Approval Gate (próximo slide) é um sinal de qualidade real, mas só aparece DEPOIS que o modelo já gerou a resposta manipulada. Um classificador de entrada pode recusar a pergunta antes de qualquer geração acontecer.

O próprio modelo barato faz o trabalho de classificar

- 02** Não precisa de um modelo dedicado extra: o mesmo Tier 1 (Módulo 4.5) classifica a pergunta como legítima ou tentativa de manipulação antes de processar — pergunta legítima segue pro RAG normalmente, tentativa de manipulação nunca chega a gerar resposta.

Testado de verdade: um ataque disfarçado passou na primeira versão

- 03** O replay do ataque da DPD foi bloqueado de primeira — mas um pedido disfarçado de "auditoria de compliance", sem as palavras que o classificador procurava, passou direto na primeira versão. A correção: parar de listar padrões de ataque e testar se a pergunta pode ser respondida citando um fato do estudo — não importa o disfarce ou a autoridade alegada.

O guardrail não substitui a observabilidade do slide anterior — os dois se somam: um pega antes de gerar, o outro mede o que passou, incluindo falso negativo do próprio guardrail.

TrialForge: Observabilidade Aplicada

Cada estudo, sua trilha de prompt versionada

- 01** Toda mudança no prompt de geração do CSR fica registrada com diff e possibilidade de replay — se um estudo reportar um documento estranho, dá pra reexecutar a mesma requisição contra a versão anterior do prompt e comparar.

Custo e uso por estudo, não só por plataforma inteira

- 02** Seguindo o padrão de hierarquia de tenant do LiteLLM — Organização → Time → Usuário — a Vitalis Platform atribui orçamento de token por estudo, com bloqueio automático se um estudo específico ultrapassar o limite.

Sinais clássicos E sinal de qualidade, lado a lado

- 03** Latência e erro por estudo continuam monitorados como em qualquer sistema distribuído — mas a plataforma também rastreia taxa de rejeição no Approval Gate ao longo do tempo, por estudo: um aumento ali é sinal de degradação que nenhum dashboard de infraestrutura mostraria.

Verificação de orçamento acontece ANTES de qualquer chamada ao modelo — se fosse depois, o gasto já teria acontecido e o controle só documentaria o estouro, sem impedir.

Você Está Vendo. Onde Isso Roda?

Observabilidade diz o que está acontecendo. Mas ainda falta decidir ONDE cada parte do sistema roda fisicamente — Kubernetes o tempo todo, Serverless sob demanda, ou mais perto do usuário, na borda. O próximo vídeo é sobre essa escolha, e sobre por que ela não é a mesma pra todo componente do TrialForge.

Módulo 5.3 → Implantação Híbrida: Kubernetes, Serverless ou Edge — a decisão não é a mesma pra todo componente.
